

- *VAX floating-point formats*—To maintain compatibility with the VAX architecture, in addition to the IEEE 754 single- and double-precision formats called S and T, Alpha supports the VAX single- and double-precision formats called F and G, but not VAX format D. (D had too narrow an exponent field to be useful for double precision and was replaced by G in VAX code.)
- *Bit count instructions*—Version 3 of the architecture added instructions to count the number of leading zeros (CTLZ), count the number of trailing zeros (CTTZ), and count the number of ones in a word (CTPOP). Originally found on Cray computers, these instructions help with decryption.

D.9

Instructions Unique to SPARC v9

Several features are unique to SPARC.

Register Windows

The primary unique feature of SPARC is register windows, an optimization for reducing register traffic on procedure calls. Several banks of registers are used, with a new one allocated on each procedure call. Although this could limit the depth of procedure calls, the limitation is avoided by operating the banks as a circular buffer, providing unlimited depth. The knee of the cost/performance curve seems to be six to eight banks.

SPARC can have between two and 32 windows, typically using eight registers each for the globals, locals, incoming parameters, and outgoing parameters. (Given that each window has 16 unique registers, an implementation of SPARC can have as few as 40 physical registers and as many as 520, although most have 128 to 136, so far.) Rather than tie window changes with call and return instructions, SPARC has the separate instructions `SAVE` and `RESTORE`. `SAVE` is used to “save” the caller’s window by pointing to the next window of registers in addition to performing an add instruction. The trick is that the source registers are from the caller’s window of the addition operation, while the destination register is in the callee’s window. SPARC compilers typically use this instruction for changing the stack pointer to allocate local variables in a new stack frame. `RESTORE` is the inverse of `SAVE`, bringing back the caller’s window while acting as an add instruction, with the source registers from the callee’s window and the destination register in the caller’s window. This automatically deallocates the stack frame. Compilers can also make use of it for generating the callee’s final return value.

The danger of register windows is that the larger number of registers could slow down the clock rate. This was not the case for early implementations. The SPARC architecture (with register windows) and the MIPS R2000 architecture (without)

have been built in several technologies since 1987. For several generations, the SPARC clock rate has not been slower than the MIPS clock rate for implementations in similar technologies, probably because cache access times dominate register access times in these implementations. The current-generation machines took different implementation strategies—in order versus out of order—and it's unlikely that the number of registers by themselves determined the clock rate in either machine. Recently, other architectures have included register windows: Tensilica and IA-64.

Another data transfer feature is alternate space option for loads and stores. This simply allows the memory system to identify memory accesses to input/output devices, or to control registers for devices such as the cache and memory management unit.

Fast Traps

Version 9 SPARC includes support to make traps fast. It expands the single level of traps to at least four levels, allowing the window overflow and underflow trap handlers to be interrupted. The extra levels mean the handler does not need to check for page faults or misaligned stack pointers explicitly in the code, thereby making the handler faster. Two new instructions were added to return from this multilevel handler: `RETRY` (which retries the interrupted instruction) and `DONE` (which does not). To support user-level traps, the instruction `RETURN` will return from the trap in nonprivileged mode.

Support for LISP and Smalltalk

The primary remaining arithmetic feature is tagged addition and subtraction. The designers of SPARC spent some time thinking about languages like LISP and Smalltalk, and this influenced some of the features of SPARC already discussed: register windows, conditional trap instructions, calls with 32-bit instruction addresses, and multiword arithmetic (see [Taylor et al. \[1986\]](#) and [Ungar et al. \[1984\]](#)). A small amount of support is offered for tagged data types with operations for addition, subtraction, and, hence, comparison. The two least significant bits indicate whether the operand is an integer (coded as 00), so `TADDcc` and `TSUBcc` set the overflow bit if either operand is not tagged as an integer or if the result is too large. A subsequent conditional branch or trap instruction can decide what to do. (If the operands are not integers, software recovers the operands, checks the types of the operands, and invokes the correct operation based on those types.) It turns out that the misaligned memory access trap can also be put to use for tagged data, since loading from a pointer with the wrong tag can be an invalid access. [Figure D.9.1](#) shows both types of tag support.

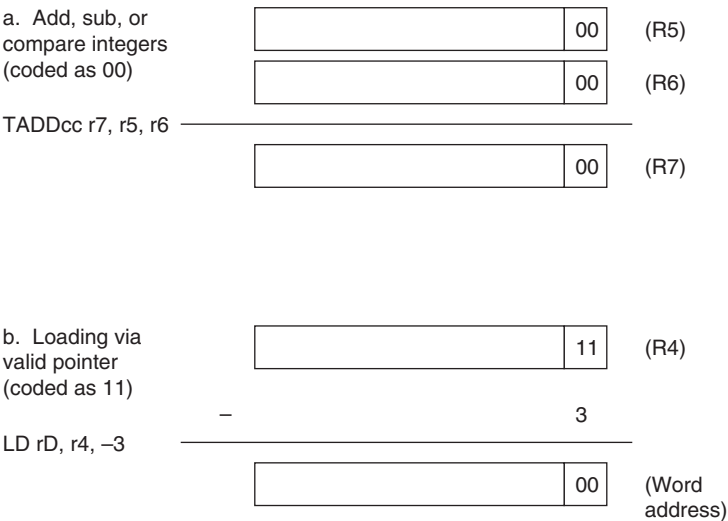


FIGURE D.9.1 SPARC uses the two least significant bits to encode different data types for the tagged arithmetic instructions. a. Integer arithmetic takes a single cycle as long as the operands and the result are integers. b. The misaligned trap can be used to catch invalid memory accesses, such as trying to use an integer as a pointer. For languages with paired data like LISP, an offset of -3 can be used to access the even word of a pair (CAR) and $+1$ can be used for the odd word of a pair (CDR).

Overlapped Integer and Floating-Point Operations

SPARC allows floating-point instructions to overlap execution with integer instructions. To recover from an interrupt during such a situation, SPARC has a queue of pending floating-point instructions and their addresses. RDPR allows the processor to empty the queue. The second floating-point feature is the inclusion of floating-point square root instructions FSQRTS, FSQRTD, and FSQRTQ.

Remaining Instructions

The remaining unique features of SPARC are as follows:

- JMWPL uses Rd to specify the return address register, so specifying r31 makes it similar to JALR in MIPS and specifying r0 makes it like JR.
- LDSTUB loads the value of the byte into Rd and then stores FF16 into the addressed byte. This version 8 instruction can be used to implement synchronization (see Chapter 2).
- CASA (CASXA) atomically compares a value in a processor register to a 32-bit (64-bit) value in memory; if and only if they are equal, it swaps the value in memory with the value in a second processor register. This version 9

instruction can be used to construct wait-free synchronization algorithms that do not require the use of locks.

- **XNOR** calculates the exclusive OR with the complement of the second operand.
- **BPcc**, **BPr**, and **FBPcc** include a branch prediction bit so that the compiler can give hints to the machine about whether a branch is likely to be taken or not.
- **ILLTRAP** causes an illegal instruction trap. [Muchnick \[1988\]](#) explains how this is used for proper execution of aggregate returning procedures in C.
- **POPC** counts the number of bits set to one in an operand, also found in the third version of the Alpha architecture.
- *Nonfaulting loads* allow compilers to move load instructions ahead of conditional control structures that control their use. Hence, nonfaulting loads will be executed speculatively.
- *Quadruple-precision floating-point arithmetic and data transfer* allow the floating-point registers to act as eight 128-bit registers for floating-point operations and data transfers.
- *Multiple-precision floating-point results for multiply* mean that two single-precision operands can result in a double-precision product and two double-precision operands can result in a quadruple-precision product. These instructions can be useful in complex arithmetic and some models of floating-point calculations.

D.10 Instructions Unique to PowerPC

PowerPC is the result of several generations of IBM commercial RISC machines—IBM RT/PC, IBM Power1, and IBM Power2—plus the Motorola 8800.

Branch Registers: Link and Counter

Rather than dedicate one of the 32 general-purpose registers to save the return address on procedure call, PowerPC puts the address into a special register called the *link register*. Since many procedures will return without calling another procedure, the link doesn't always have to be saved. Making the return address a special register makes the return jump faster, since the hardware need not go through the register read pipeline stage for return jumps.

In a similar vein, PowerPC has a *count register* to be used in *for* loops where the program iterates a fixed number of times. By using a special register, the branch

hardware can determine quickly whether a branch based on the count register is likely to branch, since the value of the register is known early in the execution cycle. Tests of the value of the count register in a branch instruction will automatically decrement the count register.

Given that the count register and link register are already located with the hardware that controls branches, and that one of the problems in branch prediction is getting the target address early in the pipeline, the PowerPC architects decided to make a second use of these registers. Either register can hold a target address of a conditional branch. Thus, PowerPC supplements its basic conditional branch with two instructions that get the target address from these registers (BCLR, BCCTR).

Remaining Instructions

Unlike most other RISC machines, register 0 is not hardwired to the value 0. It cannot be used as a base register—that is, it generates a 0 in this case—but in base + index addressing it can be used as the index. The other unique features of the PowerPC are as follows:

- *Load multiple and store multiple* save or restore up to 32 registers in a single instruction.
- LSW and STSW permit fetching and storing of fixed- and variable-length strings that have arbitrary alignment.
- *Rotate with mask* instructions support bit field extraction and insertion. One version rotates the data and then performs logical AND with a mask of ones, thereby extracting a field. The other version rotates the data but only places the bits into the destination register where there is a corresponding 1 bit in the mask, thereby inserting a field.
- *Algebraic right shift* sets the carry bit (CA) if the operand is negative and any 1 bits are shifted out. Thus, a signed divide by any constant power of two that rounds toward 0 can be accomplished with an SRAWI followed by ADDZE, which adds CA to the register.
- CBTLZ will count leading zeros.
- SUBFIC computes (immediate - RA), which can be used to develop a one's or two's complement.
- *Logical shifted immediate* instructions shift the 16-bit immediate to the left 16 bits before performing AND, OR, or XOR.

D.11 Instructions Unique to PA-RISC 2.0

PA-RISC was expanded slightly in 1990 with version 1.1 and changed significantly in 2.0 with 64-bit extensions in 1996. PA-RISC perhaps has the most unusual features of any desktop RISC machine. For example, it has the most addressing modes and instruction formats, and, as we shall see, several instructions that are really the combination of two simpler instructions.

Nullification

As shown in [Figure D.6.9](#), several RISC machines can choose not to execute the instruction following a delayed branch to improve utilization of the branch slot. This is called *nullification* in PA-RISC, and it has been generalized to apply to any arithmetic/logical instruction as well as to all branches. Thus, an `add` instruction can add two operands, store the sum, and cause the following instruction to be skipped if the sum is zero. Like conditional move instructions, nullification allows PA-RISC to avoid branches in cases where there is just one instruction in the *then* part of an *if* statement.

A Cornucopia of Conditional Branches

Given nullification, PA-RISC did not need to have separate conditional branch instructions. The inventors could have recommended that nullifying instructions precede unconditional branches, thereby simplifying the instruction set. Instead, PA-RISC has the largest number of conditional branches of any RISC machine. [Figure D.11.1](#) shows the conditional branches of PA-RISC. As you can see, several are really combinations of two instructions.

Synthesized Multiply and Divide

PA-RISC provides several primitives so that multiply and divide can be synthesized in software. Instructions that shift one operand 1, 2, or 3 bits and then add, trapping or not on overflow, are useful in multiplies. (Alpha also includes instructions that multiply the second operand of adds and subtracts by 4 or by 8: `S4ADD`, `S8ADD`, `S4SUB`, and `S8SUB`.) The divide step performs the critical step of nonrestoring divide, adding or subtracting depending on the sign of the prior result. [Magenheimer et al. \[1988\]](#) measured the size of operands in multiplies and divides to show how well the multiply step would work. Using these data for C programs, [Muchnick \[1988\]](#) found that by making special cases, the average multiply by a constant takes six clock cycles and the multiply of variables takes 24 clock cycles. PA-RISC has ten instructions for these operations.

Name	Instruction	Notation	
COMB	Compare and branch	if (cond(Rs1,Rs2))	{PC <-- PC + offset12}
COMIB	Compare immediate and branch	if (cond(imm5,Rs2))	{PC <-- PC + offset12}
MOVB	Move and branch	Rs2 <-- Rs1, if (cond(Rs1,0))	{PC <-- PC + offset12}
MOVIB	Move immediate and branch	Rs2 <-- imm5, if (cond(imm5,0))	{PC <-- PC + offset12}
ADDB	Add and branch	Rs2 <-- Rs1 + Rs2, if (cond(Rs1 + Rs2,0))	{PC <-- PC + offset12}
ADDIB	Add immediate and branch	Rs2 <-- imm5 + Rs2, if (cond(imm5 + Rs2,0))	{PC <-- PC + offset12}
BB	Branch on bit	if (cond(Rsp,0))	{PC <-- PC + offset12}
BVB	Branch on variable bit	if (cond(Rssar,0))	{PC <-- PC + offset12}

FIGURE D.11.1 The PA-RISC conditional branch instructions. The 12-bit offset is called *offset12* in this table, and the 5-bit immediate is called *imm5*. The 16 conditions are =, <, <=, odd, signed overflow, unsigned no overflow, zero or no overflow unsigned, never, and their respective complements. The BB instruction selects one of the 32 bits of the register and branches depending on whether its value is 0 or 1. The BVB selects the bit to branch using the shift amount register, a special-purpose register. The subscript notation specifies a bit field.

The original SPARC architecture used similar optimizations, but with increasing numbers of transistors the instruction set was expanded to include full multiply and divide operations. PA-RISC gives some support along these lines by putting a full 32-bit integer multiply in the floating-point unit; however, the integer data must first be moved to floating-point registers.

Decimal Operations

COBOL programs will compute on decimal values, stored as 4 bits per digit, rather than converting back and forth between binary and decimal. PA-RISC has instructions that will convert the sum from a normal 32-bit add into proper decimal digits. It also provides logical and arithmetic operations that set the condition codes to test for carries of digits, bytes, or halfwords. These operations also test whether bytes or halfwords are zero. These operations would be useful in arithmetic on 8-bit ASCII characters. Five PA-RISC instructions provide decimal support.

Remaining Instructions

Here are some remaining PA-RISC instructions:

- *Branch vectored* shifts an index register left 3 bits, adds it to a base register, and then branches to the calculated address. It is used for *case* statements.
- *Extract* and *deposit* instructions allow arbitrary bit fields to be selected from or inserted into registers. Variations include whether the extracted field is sign-extended, whether the bit field is specified directly in the instruction or indirectly in another register, and whether the rest of the register is set to zero or left unchanged. PA-RISC has 12 such instructions.

- To simplify use of 32-bit address constants, PA-RISC includes `ADDIL`, which adds a left-adjusted 21-bit constant to a register and places the result in register 1. The following data transfer instruction uses offset addressing to add the lower 11 bits of the address to register 1. This pair of instructions allows PA-RISC to add a 32-bit constant to a base register, at the cost of changing register 1.
- PA-RISC has nine debug instructions that can set breakpoints on instruction or data addresses and return the trapped addresses.
- *Load* and *clear* instructions provide a semaphore or lock that reads a value from memory and then writes zero.
- *Store bytes short* optimizes unaligned data moves, moving either the leftmost or the rightmost bytes in a word to the effective address, depending on the instruction options and condition code bits.
- Loads and stores work well with caches by having options that give hints about whether to load data into the cache if it's not already in the cache. For example, a load with a destination of register 0 is defined to be a software-controlled cache prefetch.
- PA-RISC 2.0 extended cache hints to stores to indicate block copies, recommending that the processor not load data into the cache if it's not already in the cache. It also can suggest that on loads and stores, there is spatial locality to prepare the cache for subsequent sequential accesses.
- PA-RISC 2.0 also provides an optional branch target stack to predict indirect jumps used on subroutine returns. Software can suggest which addresses get placed on and removed from the branch target stack, but hardware controls whether or not these are valid.
- *Multiply/add* and *multiply/subtract* are floating-point operations that can launch two independent floating-point operations in a single instruction in addition to the fused multiply/add and fused multiply/negate/add introduced in version 2.0 of PA-RISC.

D.12 Instructions Unique to ARM

It's hard to pick the most unusual feature of ARM, but perhaps it is the conditional execution of instructions. Every instruction starts with a 4-bit field that determines whether it will act as a nop or as a real instruction, depending on the condition codes. Hence, conditional branches are properly considered as conditionally executing the unconditional branch instruction. Conditional execution allows

avoiding a branch to jump over a single instruction. It takes less code space and time to simply conditionally execute one instruction.

The 12-bit immediate field has a novel interpretation. The 8 least significant bits are zero-extended to a 32-bit value, then rotated right the number of bits specified in the first 4 bits of the field multiplied by two. Whether this split actually catches more immediates than a simple 12-bit field would be an interesting study. One advantage is that this scheme can represent all powers of two in a 32-bit word.

Operand shifting is not limited to immediates. The second register of all arithmetic and logical processing operations has the option of being shifted before being operated on. The shift options are shift left logical, shift right logical, shift right arithmetic, and rotate right. Once again, it would be interesting to see how often operations like rotate-and-add, shift-right-and-test, and so on occur in ARM programs.

Remaining Instructions

Below is a list of the remaining unique instructions of the ARM architecture:

- *Block loads and stores*—Under control of a 16-bit mask within the instructions, any of the 16 registers can be loaded or stored into memory in a single instruction. These instructions can save and restore registers on procedure entry and return. These instructions can also be used for block memory copy—offering up to four times the bandwidth of a single register load-store—and today, block copies are the most important use.
- *Reverse subtract*—RSB allows the first register to be subtracted from the immediate or shifted register. RSC does the same thing, but includes the carry when calculating the difference.
- *Long multiplies*—Similarly to MIPS, Hi and Lo registers get the 64-bit signed product (SMULL) or the 64-bit unsigned product (UMULL).
- *No divide*—Like the Alpha, integer divide is not supported in hardware.
- *Conditional trap*—A common extension to the MIPS core found in desktop RISCs (Figures D.6.1 through D.6.4), it comes for free in the conditional execution of all ARM instructions, including SWI.
- *Coprocessor interface*—Like many of the desktop RISCs, ARM defines a full set of coprocessor instructions: data transfer, moves between general-purpose and coprocessor registers, and coprocessor operations.
- *Floating-point architecture*—Using the coprocessor interface, a floating-point architecture has been defined for ARM. It was implemented as the FPA10 coprocessor.
- *Branch and exchange instruction sets*—The BX instruction is the transition between ARM and Thumb, using the lower 31 bits of the register to set the PC and the most significant bit to determine if the mode is ARM (1) or Thumb (0).

D.13 Instructions Unique to Thumb

In the ARM version 4 model, frequently executed procedures will use ARM instructions to get maximum performance, with the less frequently executed ones using Thumb to reduce the overall code size of the program. Since typically only a few procedures dominate execution time, the hope is that this hybrid gets the best of both worlds.

Although Thumb instructions are translated by the hardware into conventional ARM instructions for execution, there are several restrictions. First, conditional execution is dropped from almost all instructions. Second, only the first eight registers are easily available in all instructions, with the stack pointer, link register, and program counter used implicitly in some instructions. Third, Thumb uses a two-operand format to save space. Fourth, the unique shifted immediates and shifted second operands have disappeared and are replaced by separate shift instructions. Fifth, the addressing modes are simplified. Finally, putting all instructions into 16 bits forces many more instruction formats.

In many ways, the simplified Thumb architecture is more conventional than ARM. Here are additional changes made from ARM in going to Thumb:

- *Drop of immediate logical instructions*—Logical immediates are gone.
- *Condition codes implicit*—Rather than have condition codes set optionally, they are defined by the opcode. All ALU instructions and none of the data transfers set the condition codes.
- *Hi/Lo register access*—The 16 ARM registers are halved into Lo registers and Hi registers, with the eight Hi registers including the stack pointer (SP), link register, and PC. The Lo registers are available in all ALU operations. Variations of ADD, BX, CMP, and MOV also work with all combinations of Lo and Hi registers. SP and PC registers are also available in variations of data transfers and add immediates. Any other operations on the Hi registers require one MOV to put the value into a Lo register, perform the operation there, and then transfer the data back to the Hi register.
- *Branch/call distance*—Since instructions are 16 bits wide, the 8-bit conditional branch address is shifted by one instead of by two. Branch with link is specified in two instructions, concatenating 11 bits from each instruction and shifting them left to form a 23-bit address to load into PC.
- *Distance for data transfer offsets*—The offset is now 5 bits for the general-purpose registers and 8 bits for SP and PC.

D.14 Instructions Unique to SuperH

Register 0 plays a special role in SuperH address modes. It can be added to another register to form an address in indirect indexed addressing and PC-relative addressing. R0 is used to load constants to give a larger addressing range than can easily be fit into the 16-bit instructions of the SuperH. R0 is also the only register that can be an operand for immediate versions of AND, CMP, OR, and XOR. Below is a list of the remaining unique details of the SuperH architecture:

- *Decrement and test*—DT decrements a register and sets the T bit to 1 if the result is 0.
- *Optional delayed branch*—Although the other embedded RISC machines generally do not use delayed branches (see [Appendix A](#)), SuperH offers optional delayed branch execution for BT and BF.
- *Many multiplies*—Depending on whether the operation is signed or unsigned, if the operands are 16 bits or 32 bits, or if the product is 32 bits or 64 bits, the proper multiply instruction is MULS, MULU, DMULS, DMULU, or MUL. The product is found in the MACL and MACH registers.
- *Zero and sign extension*—Byte or halfwords are either zero-extended (EXTU) or sign-extended (EXTS) within a 32-bit register.
- *One-bit shift amounts*—Perhaps in an attempt to make them fit within the 16-bit instructions, shift instructions only shift a single bit at a time.
- *Dynamic shift amount*—These variable shifts test the sign of the amount in a register to determine whether they shift left (positive) or shift right (negative). Both logical (SHLD) and arithmetic (SHAD) instructions are supported. These instructions help offset the 1-bit constant shift amounts of standard shifts.
- *Rotate*—SuperH offers rotations by 1 bit left (ROTL) and right (ROTR), which set the T bit with the value rotated, and also have variations that include the T bit in the rotations (ROTCL and ROTCR).
- *SWAP*—This instruction swaps either the high and low bytes of a 32-bit word or the two bytes of the rightmost 16 bits.
- *Extract word* (XTRACT)—The middle 32 bits from a pair of 32-bit registers are placed in another register.
- *Negate with carry*—Like SUBC ([Figure D.6.6](#)), except the first operand is 0.
- *Cache prefetch*—Like many of the desktop RISCs ([Figures D.6.1 through D.6.4](#)), SuperH has an instruction (PREF) to prefetch data into the cache.

- *Test-and-set*—SuperH uses the older test-and-set (TAS) instruction to perform atomic locks or semaphores (see [Chapter 2](#)). TAS first loads a byte from memory. It then sets the T bit to 1 if the byte is 0 or to 0 if the byte is not 0. Finally, it sets the most significant bit of the byte to 1 and writes the result back to memory.

D.15 Instructions Unique to M32R

The most unusual feature of the M32R is a slight VLIW approach to the pairs of 16-bit instructions. A bit is reserved in the first instruction of the pair to say whether this instruction can be executed in parallel with the next instruction—that is, the two instructions are independent—or if these two must be executed sequentially. (An earlier machine that offered a similar option was the Intel i860.) This feature is included for future implementations of the architecture.

One surprise is that all branch displacements are shifted left 2 bits before being added to the PC, and the lower 2 bits of the PC are set to 0. Since some instructions are only 16 bits long, this shift means that a branch cannot go to any instruction in the program: it can only branch to instructions on word boundaries. A similar restriction is placed on the return address for the branch-and-link and jump-and-link instructions: they can only return to a word boundary. Thus, for a slightly larger branch distance, software must ensure that all branch addresses and all return addresses are aligned to a word boundary. The M32R code space is probably slightly larger, and it probably executes more `nop` instructions than it would if the branch address was only shifted left 1 bit.

However, the VLIW feature above means that a `nop` can execute in parallel with another 16-bit instruction so that the padding doesn't take more clock cycles. The code size expansion depends on the ability of the compiler to schedule code and to pair successive 16-bit instructions; Mitsubishi claims that code size overall is only 7% larger than that for the Motorola 6800 architecture.

The last remaining novel feature is that the result of the divide operation is the remainder instead of the quotient.

D.16 Instructions Unique to MIPS-16

MIPS-16 is not really a separate instruction set but a 16-bit extension of the full 32-bit MIPS architecture. It is compatible with any of the 32-bit address MIPS architectures (MIPS I, MIPS II) or 64-bit architectures (MIPS III, IV, V). The ISA mode bit determines the width of instructions: 0 means 32-bit-wide instructions

and 1 means 16-bit-wide instructions. The new JALX instruction toggles the ISA mode bit to switch to the other ISA. JR and JALR have been redefined to set the ISA mode bit from the most significant bit of the register containing the branch address, and this bit is not considered part of the address. All jump-and-link instructions save the current mode bit as the most significant bit of the return address.

Hence, MIPS supports whole procedures containing either 16-bit or 32-bit instructions, but it does not support mixing the two lengths together in a single procedure. The one exception is the JAL and JALX: these two instructions need 32 bits even in the 16-bit mode, presumably to get a large enough address to branch to far procedures.

In picking this subset, MIPS decided to include opcodes for some three-operand instructions and to keep 16 opcodes for 64-bit operations. The combination of this many opcodes and operands in 16 bits led the architects to provide only eight easy-to-use registers—just like Thumb—whereas the other embedded RISCs offer about 16 registers. Since the hardware must include the full 32 registers of the 32-bit ISA mode, MIPS-16 includes move instructions to copy values between the eight MIPS-16 registers and the remaining 24 registers of the full MIPS architecture. To reduce pressure on the eight visible registers, the stack pointer is considered a separate register. MIPS-16 includes a variety of separate opcodes to do data transfers using SP as a base register and to increment SP: LWSP, LDSP, SWSP, SDSP, ADJSP, DADJSP, ADDIUSPD, and DADDIUSP.

To fit within the 16-bit limit, immediate fields have generally been shortened to 5 to 8 bits. MIPS-16 provides a way to extend its shorter immediates into the full width of immediates in the 32-bit mode. Borrowing a trick from the Intel 8086, the EXTEND instruction is really a 16-bit prefix that can be prepended to any MIPS-16 instruction with an address or immediate field. The prefix supplies enough bits to turn the 5-bit field of data transfers and 5- to 8-bit fields of arithmetic immediates into 16-bit constants. Alas, there are two exceptions. ADDIU and DADDIU start with 4-bit immediate fields, but since EXTEND can only supply 11 more bits, the wider immediate is limited to 15 bits. EXTEND also extends the 3-bit shift fields into 5-bit fields for shifts. (In case you were wondering, the EXTEND prefix does *not* need to start on a 32-bit boundary.)

To further address the supply of constants, MIPS-16 added a new addressing mode! PC-relative addressing for load word (LWPC) and load double (LDPC) shifts an 8-bit immediate field by 2 or 3 bits, respectively, adding it to the PC with the lower 2 or 3 bits cleared. The constant word or doubleword is then loaded into a register. Thus 32-bit or 64-bit constants can be included with MIPS-16 code, despite the loss of LIU to set the upper register bits. Given the new addressing mode, there is also an instruction (ADDIUPC) to calculate a PC-relative address and place it in a register.

MIPS-16 differs from the other embedded RISCs in that it can subset a 64-bit address architecture. As a result it has 16-bit instruction-length versions of 64-bit

data operations: data transfer (LD, SD, LWU), arithmetic operations (DADDU/IU, DSUBU, DMULT/U, DDIV/U), and shifts (DSLL/V, DSRA/V, DSRL/V).

Since MIPS plays such a prominent role in this book, we show all the additional changes made from the MIPS core instructions in going to MIPS-16:

- *Drop of signed arithmetic instructions*—Arithmetic instructions that can trap were dropped to save opcode space: ADD, ADDI, SUB, DADD, DADDI, DSUB.
- *Drop of immediate logical instructions*—Logical immediates are gone too: ANDI, ORI, XORI.
- *Branch instructions pared down*—Comparing two registers and then branching did not fit, nor did all the other comparisons of a register to zero. Hence these instructions didn't make it either: BEQ, BNE, BGEZ, BGTZ, BLEZ, and BLTZ. As mentioned in Section D.3, to help compensate MIPS-16 includes compare instructions to test if two registers are equal. Since compare and set on less than set the new T register, branches were added to test the T register.
- *Branch distance*—Since instructions are 16 bits wide, the branch address is shifted by one instead of by two.
- *Delayed branches disappear*—The branches take effect before the next instruction. Jumps still have a one-slot delay.
- *Extension and distance for data transfer offsets*—The 5-bit and 8-bit fields are zero-extended instead of sign-extended in 32-bit mode. To get greater range, the immediate fields are shifted left 1, 2, or 3 bits depending on whether the data are halfword, word, or doubleword. If the EXTEND prefix is prepended to these instructions, they use the conventional signed 16-bit immediate of the 32-bit mode.
- *Extension of arithmetic immediates*—The 5-bit and 8-bit fields are zero-extended for set on less than and compare instructions, for forming a PC-relative address, and for adding to SP and placing the result in a register (ADDIUSP, DADDIUSP). Once again, if the EXTEND prefix is prepended to these instructions, they use the conventional signed 16-bit immediate of the 32-bit mode. They are still sign-extended for general adds and for adding to SP and placing the result back in SP (ADJSP, DADJSP). Alas, code density and orthogonality are strange bedfellows in MIPS-16!
- *Redefining shift amount of 0*—MIPS-16 defines the value 0 in the 3-bit shift field to mean a shift of 8 bits.
- *New instructions added due to loss of register 0 as zero*—Load immediate, negate, and not were added, since these operations could no longer be synthesized from other instructions using r0 as a source.

D.17 Concluding Remarks

This appendix covers the addressing modes, instruction formats, and all instructions found in 10 RISC architectures. Although the later sections of the appendix concentrate on the differences, it would not be possible to cover 10 architectures in these few pages if there were not so many similarities. In fact, we would guess that more than 90% of the instructions executed for any of these architectures would be found in [Figures D.3.5 through D.3.11](#). To contrast this homogeneity, [Figure D.17.1](#) gives a summary for four architectures from the 1970s in a format similar to that shown in [Figure D.1.1](#). (Imagine trying to write a single chapter in this style for those architectures!) In the history of computing, there has never been such widespread agreement on computer architecture.

	IBM 360/370	Intel 8086	Motorola 68000	DEC VAX
Date announced	1964/1970	1978	1980	1977
Instruction size(s) (bits)	16, 32, 48	8, 16, 24, 32, 40, 48	16, 32, 48, 64, 80	8, 16, 24, 32, ..., 432
Addressing (size, model)	24 bits, flat/31 bits, flat	4 + 16 bits, segmented	24 bits, flat	32 bits, flat
Data aligned?	Yes 360/No 370	No	16-bit aligned	No
Data addressing modes	2/3	5	9	= 14
Protection	Page	None	Optional	Page
Page size	2 KB & 4 KB	—	0.25 to 32 KB	0.5 KB
I/O	Opcode	Opcode	Memory mapped	Memory mapped
Integer registers (size, model, number)	16 GPR × 32 bits	8 dedicated data × 16 bits	8 data and 8 address × 32 bits	15 GPR × 32 bits
Separate floating-point registers	4 × 64 bits	Optional: 8 × 80 bits	Optional: 8 × 80 bits	0
Floating-point format	IBM (floating hexadecimal)	IEEE 754 single, double, extended	IEEE 754 single, double, extended	DEC

FIGURE D.17.1 Summary of four 1970s architectures. Unlike the architectures in [Figure D.1.1](#), there is little agreement between these architectures in any category.

This style of architecture cannot remain static, however. Like people, instruction sets tend to get bigger as they get older. [Figure D.17.2](#) shows the genealogy of these instruction sets, and [Figure D.17.3](#) shows which features were added to or deleted from generations of desktop RISCs over time.

As you can see, all the desktop RISC machines have evolved to 64-bit address architectures, and they have done so fairly painlessly.

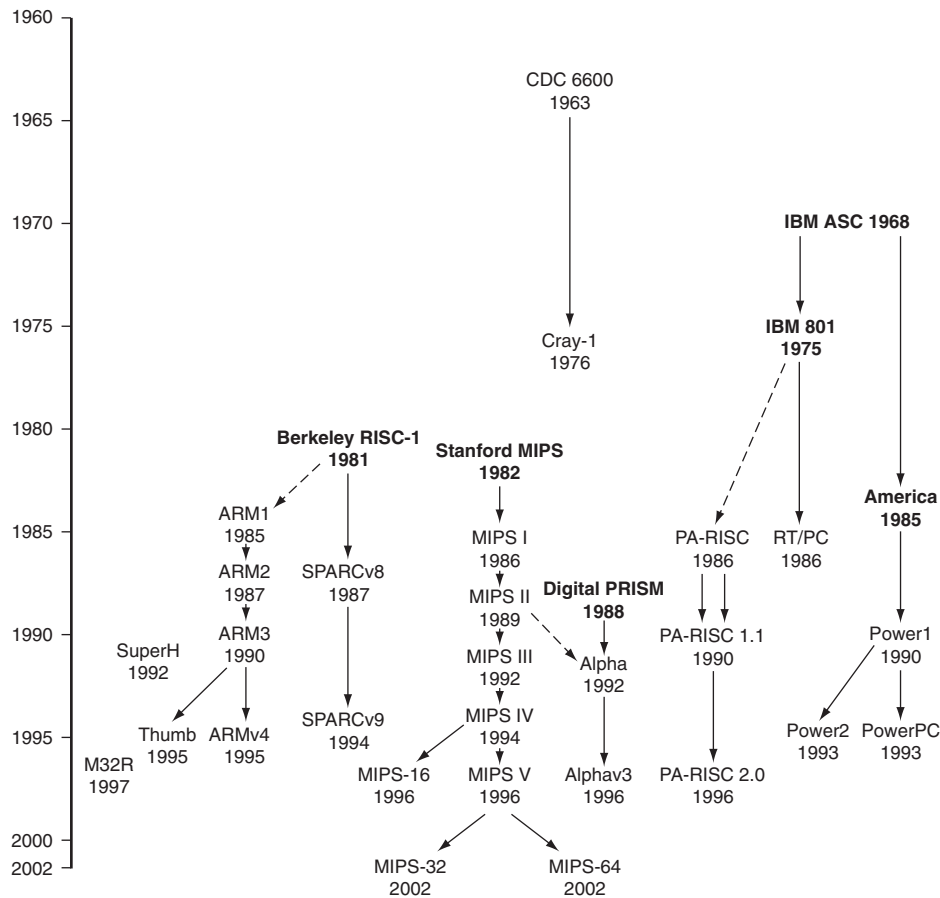


FIGURE D.17.2 The lineage of RISC instruction sets. Commercial machines are shown in plain text and research machines in bold. The CDC 6600 and Cray-1 were load-store machines with register 0 fixed at 0, and with separate integer and floating-point registers. Instructions could not cross word boundaries. An early IBM research machine led to the 801 and America research projects, with the 801 leading to the unsuccessful RT/PC and America leading to the successful Power architecture. Some people who worked on the 801 later joined Hewlett-Packard to work on the PA-RISC. The two university projects were the basis of MIPS and SPARC machines. According to Furber [1996], the Berkeley RISC project was the inspiration of the ARM architecture. While ARM1, ARM2, and ARM3 were names of both architectures and chips, ARM version 4 is the name of the architecture used in ARM7, ARM8, and StrongARM chips. (There are no ARMv4 and ARM5 chips, but ARM6 and early ARM7 chips use the ARM3 architecture.) DEC built a RISC microprocessor in 1988 but did not introduce it. Instead, DEC shipped workstations using MIPS microprocessors for 3 years before they brought out their own RISC instruction set, Alpha 21064, which is very similar to MIPS III and PRISM. The Alpha architecture has had small extensions, but they have not been formalized with version numbers; we used version 3 because that is the version of the reference manual. The Alpha 21164A chip added byte and halfword loads and stores, and the Alpha 21264 includes the MAX multimedia and bit count instructions. Internally, Digital names chips after the fabrication technology: EV4 (21064), EV45 (21064A), EV5 (21164), EV56 (21164A), and EV6 (21264). “EV” stands for “extended VAX.”

Feature	PA-RISC			SPARC		MIPS					Power		
	1.0	1.1	2.0	v8	v9	I	II	III	IV	V	1	2	PC
Interlocked loads	X	"	"	X	"		+	"	"		X	"	"
Load-store FP double	X	"	"	X	"		+	"	"		X	"	"
Semaphore	X	"	"	X	"		+	"	"		X	"	"
Square root	X	"	"	X	"		+	"	"			+	"
Single-precision FP ops	X	"	"	X	"	X	"	"	"				+
Memory synchronize	X	"	"	X	"		+	"	"		X	"	"
Coprocessor	X	"	"	X	—	X	"	"	"				
Base + index addressing	X	"	"	X	"				+		X	"	"
Equiv. 32 64-bit FP registers		"	"		+			+	"		X	"	"
Annulling delayed branch	X	"	"	X	"		+	"	"				
Branch register contents	X	"	"		+	X	"	"	"				
Big/little endian		+	"		+	X	"	"	"				+
Branch prediction bit					+		+	"	"		X	"	"
Conditional move					+				+		X	"	—
Prefetch data into cache			+		+				+		X	"	"
64-bit addressing/int. ops			+		+			+	"				+
32-bit multiply, divide		+	"		+	X	"	"	"		X	"	"
Load-store FP quad					+							+	—
Fused FP mul/add			+						+		X	"	"
String instructions	X	"	"								X	"	—
Multimedia support		X	"	X						X			

FIGURE D.17.3 Features added to desktop RISC machines. X means in the original machine, + means added later, " means continued from prior machine, and — means removed from architecture. Alpha is not included, but it added byte and word loads and stores, and bit count and multimedia extensions, in version 3. MIPS V added the MDMX instructions and paired single floating-point operations.

We would like to thank the following people for comments on drafts of this appendix: Professor Steven B. Furber, University of Manchester; Dr. Dileep Bhandarkar, Intel Corporation; Dr. Earl Killian, Silicon Graphics/MIPS; and Dr. Hiokazu Takata, Mitsubishi Electric Corporation.

Further Reading

Bhandarkar, D. P. [1995]. *Alpha Architecture and Implementations*, Newton, MA: Digital Press.

Darcy, J.D., and D. Gay [1996]. "FLECKmarks: Measuring floating point performance using a full IEEE compliant arithmetic benchmark," CS 252 class project, U.C. Berkeley (see www.sonic.net/~jddarcy/Research/fleckmrk.pdf).

Digital Semiconductor [1996]. *Alpha Architecture Handbook*, Version 3, Maynard, MA: Digital Press, Order number EC-QD2KB-TE (October).

- Furber, S. B. [1996]. *ARM System Architecture*, Harlow, England: Addison-Wesley. (See http://www.pearsonhighered.com/pearsonhigheredus/educator/product/products_detail.page?isbn=9780201675191&forced_logout=forced_logged_out#sthash.QX4WfErc).
- Hewlett-Packard [1994]. *PA-RISC 2.0 Architecture Reference Manual*, 3rd ed.
- Hitachi [1997]. *SuperH RISC Engine SH7700 Series Programming Manual*. (See <http://am.renesas.com/products/mpumcu/superh/sh7700/Documentation.jsp>).
- IBM [1994]. *The PowerPC Architecture*, San Francisco: Morgan Kaufmann.
- Kane, G. [1996]. *PA-RISC 2.0 Architecture*, Upper Saddle River, NJ: Prentice Hall PTR.
- Kane, G. and J. Heinrich [1992]. *MIPS RISC Architecture*, Englewood Cliffs, NJ: Prentice Hall.
- Kissell, K.D. [1997]. *MIPS16: High-Density for the Embedded Market*.
- Magenheimer, D. J., L. Peters, K. W. Pettis, and D. Zuras [1988]. "Integer multiplication and division on the HP precision architecture", *IEEE Trans. on Computers* 37:8, 980–90.
- MIPS [1997]. *MIPS16 Application Specific Extension Product Description*.
- Mitsubishi [1996]. *Mitsubishi 32-Bit Single Chip Microcomputer M32R Family Software Manual* (September).
- Muchnick, S. S. [1988]. "Optimizing compilers for SPARC", *Sun Technology* 1:3 (Summer), 64–77.
- Seal, D. *Arm Architecture Reference Manual*, 2nd ed, Morgan Kaufmann, 2000.
- Silicon Graphics [1996]. *MIPS V Instruction Set*.
- Sites, R. L., and R. Witek (eds.) [1995]. *Alpha Architecture Reference Manual*, 2nd ed. Newton, MA: Digital Press.
- Sloss, A. N., D. Symes, and C. Wright, *ARM System Developer's Guide* San Francisco: Elsevier Morgan Kaufmann, 2004.
- Sun Microsystems [1989]. *The SPARC Architectural Manual*, Version 8, Part No. 800-1399-09, August 25.
- Sweetman, D. *See MIPS Run*, 2nd ed, Morgan Kaufmann, 2006.
- Taylor, G., P. Hilfinger, J. Larus, D. Patterson, and B. Zorn [1986]. "Evaluation of the SPUR LISP architecture," *Proc. 13th Symposium on Computer Architecture* (June), Tokyo.
- Ungar, D., R. Blau, P. Foley, D. Samples, and D. Patterson [1984]. "Architecture of SOAR: Smalltalk on a RISC," *Proc. 11th Symposium on Computer Architecture* (June), Ann Arbor, MI, 188–97.
- Weaver, D. L. and T. Germond [1994]. *The SPARC Architectural Manual*, Version 9 Englewood Cliffs, NJ: Prentice Hall.
- Weiss, S. and J. E. Smith [1994]. *Power and PowerPC*, San Francisco: Morgan Kaufmann.

Answers to Check Yourself

Chapter 1

§1.1, page 10: Discussion questions: many answers are acceptable.

§1.4, page 24: DRAM memory: volatile, short access time of 50 to 70 nanoseconds, and cost per GB is \$5 to \$10. Disk memory: nonvolatile, access times are 100,000 to 400,000 times slower than DRAM, and cost per GB is 100 times cheaper than DRAM. Flash memory: nonvolatile, access times are 100 to 1000 times slower than DRAM, and cost per GB is 7 to 10 times cheaper than DRAM.

§1.5, page 28: 1, 3, and 4 are valid reasons. Answer 5 can be generally true because high volume can make the extra investment to reduce die size by, say, 10% a good economic decision, but it doesn't have to be true.

§1.6, page 33: 1. a: both, b: latency, c: neither. 7 seconds.

§1.6, page 40: b.

§1.10, page 51: a. Computer A has the higher MIPS rating. b. Computer B is faster.

Chapter 2

§2.2, page 67: ARMv8, C, Java.

§2.3, page 73: 2) Very slow.

§2.4, page 81: 2) – 8ten

§2.5, page 89: 3) SUB X11, X10, X9

§2.6, page 92: Both. AND with a mask pattern of 1s will leaves 0s everywhere but the desired field. Shifting left by the correct amount removes the bits from the left of the field. Shifting right by the appropriate amount puts the field into the rightmost bits of the doubleword, with 0s in the rest of the doubleword. Note that AND leaves the field where it was originally, and the shift pair moves the field into the rightmost part of the doubleword.

§2.7, page 99: I. All are true. II. 1).

§2.8, page 110: Both are true.

§2.9, page 115: I. 1) and 2) II. 3).

§2.10, page 124: I. 4) $\pm 1024\text{K}$. II. 4) $\pm 128\text{M}$.

§2.11, page 128: Both are true.

§2.12, page 137: 4) Machine independence.

Chapter 3

§3.2, page 191: 2.

§3.5, page 229: 3.

Chapter 4

§4.1, page 260: 3 of 5: Control, Datapath, Memory. Input and Output are missing.

§4.2, page 263: false. Edge-triggered state elements make simultaneous reading and writing both possible and unambiguous.

§4.3, page 270: I. a. II. c.

§4.4, page 283: Yes, Branch and ALUOp0 are identical. In addition, you can use the flexibility of the don't care bits to combine other signals together. ALUSrc and MemtoReg can be made the same by setting the two don't care bits of MemtoReg to 1 and 0. Reg2Loc and RegWrite can be made to be inverses of one another by setting the don't care bit of Reg2Loc to 0. You don't need an inverter; simply use the other signal and flip the order of the inputs to the Reg2Loc multiplexor!

§4.5, page 296: 1. Stall due to a load-use data hazard of the LDUR result. II. Avoid stalling in the third instruction for the read-after-write data hazard on X1 by forwarding the ADD result. 3. It need not stall, even without forwarding.

§4.6, page 309: Statements 2 and 4 are correct; the rest are incorrect.

§4.8, page 335: 1. Predict not taken. 2. Predict taken. 3. Dynamic prediction.

§4.9, page 342: The first instruction, since it is logically executed before the others.

§4.10, page 355: 1. Both. 2. Both. 3. Software. 4. Hardware. 5. Hardware. 6. Hardware. 7. Both. 8. Hardware. 9. Both.

§4.12, page 365: First two are false and the last two are true.

§4.13, page 4.13-4: Only statement #3 is completely accurate

§4.13, page 4.13-6: Statements #1 and #4 are true

Chapter 5

§5.1, page 391: 1 and 4. (3 is false because the cost of the memory hierarchy varies per computer, but in 2016 the highest cost is usually the DRAM.)

§5.3, page 412: 1 and 4: A lower miss penalty can enable smaller blocks, since you don't have that much latency to amortize, yet higher memory bandwidth usually leads to larger blocks, since the miss penalty is only slightly larger.

§5.4, page 431: 1.

§5.7, page 465: 1-a, 2-c, 3-b, 4-d.

§5.8, page 472: 2. (Both large block sizes and prefetching may reduce compulsory misses, so 1 is false.)

§5.11, page 5.11-8: All are true.

Chapter 6

§6.1, page 518: False. Task-level parallelism can help sequential applications and sequential applications can be made to run on parallel hardware, although it is more challenging.

§6.2, page 523: False. *Weak* scaling can compensate for a serial portion of the program that would otherwise limit scalability, but not so for strong scaling.

§6.3, page 528: True, but they are missing useful vector features like gather-scatter and vector length registers that improve the efficiency of vector architectures. (As an elaboration in this section mentions, the AVX2 SIMD extensions offers indexed loads via a gather operation but *not* scatter for indexed stores. The Haswell generation x86 microprocessor is the first to support AVX2.)

§6.4, page 533: 1. True. 2. True.

§6.5, page 537: False. Since the shared address is a *physical* address, multiple tasks each in their own *virtual* address spaces can run well on a shared memory multiprocessor.

§6.6, page 545: False. Graphics DRAM chips are prized for their higher bandwidth.

§6.7, page 550: 1. False. Sending and receiving a message is an implicit synchronization, as well as a way to share data. 2. True.

§6.8, page 552: True.

§6.10, page 564: True. We likely need innovation at all levels of the hardware and software stack for parallel computing to succeed.

§6.9, page 6.9-10: 1. polling with loads and stores. 2. interrupts with DMA.

This page intentionally left blank

Glossary

absolute address A variable's or routine's actual address in memory.

abstraction A model that renders lower-level details of computer systems temporarily invisible to facilitate the design of sophisticated systems.

access bit Also called **use bit** or **reference bit**. A field that is set whenever a page is accessed and that is used to implement LRU or other replacement schemes.

acronym A word constructed by taking the initial letters of a string of words. For example: **RAM** is an acronym for Random Access Memory, and **CPU** is an acronym for Central Processing Unit.

active matrix display A liquid crystal display using a transistor to control the transmission of light at each individual pixel.

address A value used to delineate the location of a specific data element within a memory array.

address translation Also called **address mapping**. The process by which a virtual address is mapped to an address used to access memory.

addressing mode One of the several addressing regimes delimited by their varied use of operands and/or addresses.

aliasing A situation in which two addresses access the same object; it can occur in virtual memory when there are two virtual addresses for the same physical page.

alignment restriction A requirement that data be aligned in memory on natural boundaries.

Amdahl's Law A rule stating that the performance enhancement possible with a given improvement is limited by the amount that the improved feature is used. It is a quantitative version of the law of diminishing returns.

AND A logical bit-by-bit operation with two operands that calculates a 1 only if there is a 1 in *both* operands.

antidependence Also called **name dependence**. An ordering forced by the reuse of a name, typically a register, rather than by a true dependence that carries a value between two instructions.

antifuse A structure in an integrated circuit that when programmed makes a permanent connection between two wires.

application binary interface (ABI) The user portion of the instruction set plus the operating system interfaces used by application programmers. It defines a standard for binary portability across computers.

architectural registers The instruction set of visible registers of a processor; for example, in LEGv8, these are the 32 integer and 32 floating-point registers.

arithmetic intensity The ratio of floating-point operations in a program to the number of data bytes accessed by a program from main memory.

arithmetic logic unit (ALU) Hardware that performs addition, subtraction, and usually logical operations such as AND and OR.

assembler A program that translates a symbolic version of instruction into the binary version.

assembler directive An operation that tells the assembler how to translate a program but does not produce machine instructions; always begins with a period.

assembly language A symbolic language that can be translated into binary machine language.

asserted The signal is logically high or true.

asserted signal A signal that is (logically) true, or 1.

backpatching A method for translating from assembly language to machine instructions in which the assembler builds a (possibly incomplete) binary representation of every instruction in one pass over a program and then returns to fill in previously undefined labels.

basic block A sequence of instructions without branches (except possibly at the end) and without branch targets or branch labels (except possibly at the beginning).

behavioral specification Describes how a digital system operates functionally.

benchmark A program selected for use in comparing computer performance.

biased notation A notation that represents the most negative value by $00\dots000_{\text{two}}$ and the most positive value by $11\dots11_{\text{two}}$, with 0 typically having the value $10\dots00_{\text{two}}$, thereby biasing the number such that the number plus the bias has a nonnegative representation.

binary digit Also called **binary bit**. One of the two numbers in base 2, 0 or 1, that are the components of information.

bisection bandwidth The bandwidth between two equal parts of a multiprocessor. This measure is for a worst-case split of the multiprocessor.

block (or line) The minimum unit of information that can be either present or not present in a cache.

blocking assignment In Verilog, an assignment that completes before the execution of the next statement.

branch address table Also called **branch table**. A table of addresses of alternative instruction sequences.

branch-and-link instruction An instruction that branches to an address and simultaneously saves the address of the following instruction in a register (LR or X30 in LEGv8).

branch not taken or (untaken branch) A branch where the branch condition is false and the program counter (PC) becomes the address of the instruction that sequentially follows the branch.

branch prediction A method of resolving a branch hazard that assumes a given outcome for the conditional branch and proceeds from that assumption rather than waiting to ascertain the actual outcome.

branch prediction buffer Also called **branch history table**. A small memory that is indexed by the lower portion of the address of the branch instruction and that contains one or more bits indicating whether the branch was recently taken or not.

branch taken A branch where the branch condition is satisfied and the program counter (PC) becomes the branch target. All unconditional branches are taken branches.

branch target address The address specified in a branch, which becomes the new program counter (PC) if the branch is taken. In the ARMv8 architecture, the branch target is given by the sum of the offset field of the instruction and the address of the branch.

branch target buffer A structure that caches the destination PC or destination instruction for a branch. It is usually organized as a cache with tags, making it more costly than a simple prediction buffer.

bus In logic design, a collection of data lines that are treated together as a single logical signal; also, a shared collection of lines with multiple sources and uses.

cache memory A small, fast memory that acts as a buffer for a slower, larger memory.

cache miss A request for data from the cache that cannot be filled because the data are not present in the cache.

callee A procedure that executes a series of stored instructions based on parameters provided by the caller and then returns control to the caller.

callee-saved register A register saved by the routine making a procedure call.

caller The program that instigates a procedure and provides the necessary parameter values.

caller-saved register A register saved by the routine being called.

capacity miss A cache miss that occurs because the cache, even with full associativity, cannot contain all the blocks needed to satisfy the request.

central processing unit (CPU) Also called central processor unit or processor. The active part of the computer, which contains the datapath and control and which adds numbers, tests numbers, signals I/O devices to activate, and so on.

clock cycle Also called **tick**, **clock tick**, **clock period**, **clock**, or **cycle**. The time for one clock period, usually of the processor clock.

clock cycles per instruction (CPI) Average number of clock cycles per instruction for a program or program fragment.

clock period The length of each clock cycle.

clock skew The difference in absolute time between the times when two state elements see a clock edge.

clocking methodology The approach used to determine when data are valid and stable relative to the clock.

Cloud Computing refers to large collections of servers that provide services over the Internet; some providers rent dynamically varying numbers of servers as a utility.

cluster A set of computers connected over a local area network that function as a single large multiprocessor.

clusters Collections of computers connected via I/O over standard network switches to form a message-passing multiprocessor.

coarse-grained multithreading A version of hardware multithreading that implies switching between threads only after significant events, such as a last-level cache miss.

combinational element An operational element, such as an AND gate or an ALU.

combinational logic A logic system whose blocks do not contain memory and hence compute the same output given the same input.

commit unit The unit in a dynamic or out-of-order execution pipeline that decides when it is safe to release the result of an operation to programmer-visible registers and memory.

compiler A program that translates high-level language statements into assembly language statements.

compulsory miss Also called **cold-start miss**. A cache miss caused by the first access to a block that has never been in the cache.

conditional branch An instruction that tests a value, and that allows for a subsequent transfer of control to a new address in the program based on the outcome of the test.

conflict miss Also called **collision miss**. A cache miss that occurs in a set-associative or direct-mapped cache when multiple blocks compete for the same set and that are eliminated in a fully associative cache of the same size.

context switch A changing of the internal state of the processor to allow a different process to use the processor that includes saving the state needed to return to the currently executing process.

control The component of the processor that commands the datapath, memory, and I/O devices according to the instructions of the program.

control hazard Also called **branch hazard**. Arises when the proper instruction cannot execute in the proper pipeline clock cycle because the instruction that was fetched is not the one that is needed; that is, the flow of instruction addresses is not what the pipeline expected.

control signal A signal used for multiplexor selection or for directing the operation of a functional unit; contrasts with a **data signal**, which contains information that is operated on by a functional unit.

correlating predictor A branch predictor that combines local behavior of a particular branch and global information about the behavior of some recent number of executed branches.

CPU execution time Also called **CPU time**. The actual time the CPU spends computing for a specific task.

crossbar network A network that allows any node to communicate with any other node in one pass through the network.

D flip-flop A flip-flop with one data input that stores the value of that input signal in the internal memory when the clock edge occurs.

data hazard Also called a **pipeline data hazard**. When a planned instruction cannot execute in the proper clock cycle because data that are needed to execute the instruction are not yet available.

data race Two memory accesses forming a data race if they are from different threads to the same location, at least one is a write, and they occur one after another.

data segment The segment of a UNIX object or executable file that contains a binary representation of the initialized data used by the program.

data transfer instruction A command that moves data between memory and registers.

data-level parallelism Parallelism achieved by performing the same operation on independent data.

datapath The component of the processor that performs arithmetic operations.

datapath element A unit used to operate on or hold data within a processor. In the LEGv8 implementation, the datapath elements include the instruction and data memories, the register file, the ALU, and adders.

deasserted The signal being logically low or false.

deasserted signal A signal that is (logically) false, or 0.

decoder A logic block that has an n -bit input and $2n$ outputs, where only one output is asserted for each input combination.

defect A microscopic flaw in a wafer or in patterning steps that can result in the failure of the die containing that defect.

delayed branch A type of branch where the instruction immediately following the

branch is always executed, independent of whether the branch condition is true or false.

die The individual rectangular sections that are cut from a wafer, more informally known as **chips**.

direct-mapped cache A cache structure in which each memory location is mapped to exactly one location in the cache.

dividend A number being divided.

divisor A number that the dividend is divided by.

don't-care term An element of a logical function in which the output does not depend on the values of all the inputs. Don't-care terms may be specified in different ways.

double precision A floating-point value represented in a 64-bit doubleword.

doubleword Another natural unit of access in a computer, usually a group of 64 bits; corresponds to the size of a register in the LEGv8 architecture.

dynamic branch prediction Prediction of branches at runtime using runtime information.

dynamic multiple issue An approach to implementing a multiple-issue processor where many decisions are made during execution by the processor.

dynamic pipeline scheduling Hardware support for reordering the order of instruction execution so as to avoid stalls.

dynamic random access memory (DRAM) Memory built as an integrated circuit; it provides random access to any location. Access times are 50 nanoseconds and cost per gigabyte in 2012 was \$5 to \$10.

dynamically linked libraries (DLLs) Library routines that are linked to a program during execution.

edge-triggered clocking A clocking scheme in which all state changes occur on a clock edge.

embedded computer A computer inside another device used for running one predetermined application or collection of software.

EOR A logical bit-by-bit operation with two operands that calculates the exclusive OR of the two operands. That is, it calculates a 1 only if the values are different in the two operands.

error detection code A code that enables the detection of an error in data, but not the precise location and, hence, correction of the error.

exception Also called an **interrupt**. An unscheduled event that disrupts program execution; used to detect overflow.

exception enable Also called interrupt enable. A signal or action that controls whether the process responds to an exception or not; necessary for preventing the occurrence of exceptions during intervals before the processor has safely saved the state needed to restart.

executable file A functional program in the format of an object file that contains no unresolved references. It can contain symbol tables and debugging information. A “stripped executable” does not contain that information. Relocation information may be included for the loader.

exponent In the numerical representation system of floating-point arithmetic, the value that is placed in the exponent field.

external label Also called **global label**. A label referring to an object that can be referenced from files other than the one in which it is defined.

false sharing When two unrelated shared variables are located in the same cache block and the full block is exchanged between processors even though the processors are accessing different variables.

field programmable devices (FPD) An integrated circuit containing combinational logic, and possibly memory devices, that are configurable by the end user.

field programmable gate array (FPGA) A configurable integrated circuit containing both combinational logic blocks and flip-flops.

fine-grained multithreading A version of hardware multithreading that implies

switching between threads after every instruction.

finite-state machine A sequential logic function consisting of a set of inputs and outputs, a next-state function that maps the current state and the inputs to a new state, and an output function that maps the current state and possibly the inputs to a set of asserted outputs.

flash memory A nonvolatile semiconductor memory. It is cheaper and slower than DRAM but more expensive per bit and faster than magnetic disks. Access times are about 5 to 50 microseconds and cost per gigabyte in 2012 was \$0.75 to \$1.00.

flip-flop A memory element for which the output is equal to the value of the stored state inside the element and for which the internal state is changed only on a clock edge.

floating point Computer arithmetic that represents numbers in which the binary point is not fixed.

flush To discard instructions in a pipeline, usually due to an unexpected event.

formal parameter A variable that is the argument to a procedure or macro; replaced by that argument once the macro is expanded.

forward reference A label that is used before it is defined.

forwarding Also called **bypassing**. A method of resolving a data hazard by retrieving the missing data element from internal buffers rather than waiting for it to arrive from programmer-visible registers or memory.

fraction The value, generally between 0 and 1, placed in the fraction field.

frame pointer A value denoting the location of the saved registers and local variables for a given procedure.

fully associative cache A cache structure in which a block can be placed in any location in the cache.

fully connected network A network that connects processor-memory nodes by supplying a dedicated communication link between every node.

fused multiply add A floating-point instruction that performs both a multiply and an add, but rounds only once after the add.

gate A device that implements basic logic functions, such as AND or OR.

global miss rate The fraction of references that miss in all levels of a multilevel cache.

global pointer The register that is reserved to point to the static area.

guard The first of two extra bits kept on the right during intermediate calculations of floating-point numbers; used to improve rounding accuracy.

handler Name of a software routine invoked to “handle” an exception or interrupt.

hardware description language A programming language for describing hardware, used for generating simulations of a hardware design and also as input to synthesis tools that can generate actual hardware.

hardware multithreading Increasing utilization of a processor by switching to another thread when one thread is stalled.

hardware synthesis tools Computer-aided design software that can generate a gate-level design based on behavioral descriptions of a digital system.

hexadecimal Numbers in base 16.

high-level programming language A portable language such as C, C++, Java, or Visual Basic that is composed of words and algebraic notation that can be translated by a compiler into assembly language.

hit rate The fraction of memory accesses found in a level of the memory hierarchy.

hit time The time required to access a level of the memory hierarchy, including the time needed to determine whether the access is a hit or a miss.

hold time The minimum time during which the input must be valid after the clock edge.

implementation Hardware that obeys the architecture abstraction.

imprecise interrupt Also called **imprecise exception**. Interrupts or exceptions in pipelined computers that are not associated with the exact instruction that was the cause of the interrupt or exception.

in-order commit A commit in which the results of pipelined execution are written to the programmer visible state in the same order that instructions are fetched.

input device A mechanism through which the computer is fed information, such as a microphone.

instruction A command that computer hardware understands and obeys.

instruction count The number of instructions executed by the program.

instruction format A form of representation of an instruction composed of fields of binary numbers.

instruction latency The inherent execution time for an instruction.

instruction-level parallelism The parallelism among instructions.

instruction mix A measure of the dynamic frequency of instructions across one or many programs.

instruction set architecture Also called **architecture**. An abstract interface between the hardware and the lowest-level software that encompasses all the information necessary to write a machine language program that will run correctly, including instructions, registers, memory access, I/O, and so on.

integrated circuit Also called a **chip**. A device combining dozens to millions of transistors.

interrupt An exception that comes from outside of the processor. (Some architectures use the term *interrupt* for all exceptions.)

interrupt handler A piece of code that is run as a result of an exception or an interrupt.

issue packet The set of instructions that issues together in one clock cycle; the packet may be determined statically by the compiler or dynamically by the processor.

issue slots The positions from which instructions could issue in a given clock cycle; by analogy, these correspond to positions at the starting blocks for a sprint.

Java bytecode Instruction from an instruction set designed to interpret Java programs.

Just In Time compiler (JIT) The name commonly given to a compiler that operates at runtime, translating the interpreted code segments into the native code of the computer.

latch A memory element in which the output is equal to the value of the stored state inside the element and the state is changed whenever the appropriate inputs change and the clock is asserted.

latency (pipeline) The number of stages in a pipeline or the number of stages between two instructions during execution.

least recently used (LRU) A replacement scheme in which the block replaced is the one that has been unused for the longest time.

least significant bit The rightmost bit in a LEGv8 doubleword.

level-sensitive clocking A timing methodology in which state changes occur at either high or low clock levels but are not instantaneous, as such changes are in edge-triggered designs.

linker Also called **link editor**. A systems program that combines independently assembled machine language programs and resolves all undefined labels into an executable file.

liquid crystal display A display technology using a thin layer of liquid polymers that can be used to transmit or block light according to whether a charge is applied.

load-use data hazard A specific form of data hazard in which the data being loaded by a load instruction have not yet become available when they are needed by another instruction.

loader A systems program that places an object program in main memory so that it is ready to execute.

local area network (LAN) A network designed to carry data within a geographically confined area, typically within a single building.

local label A label referring to an object that can be used only within the file in which it is defined.

local miss rate The fraction of references to one level of a cache that miss; used in multilevel hierarchies.

lock A synchronization device that allows access to data to only one processor at a time.

lookup tables (LUTs) In a field programmable device, the name given to the cells because they consist of a small amount of logic and RAM.

loop unrolling A technique to get more performance from loops that access arrays, in which multiple copies of the loop body are made and instructions from different iterations are scheduled together.

machine language Binary representation used for communication within a computer system.

macro A pattern-matching and replacement facility that provides a simple mechanism to name a frequently used sequence of instructions.

magnetic disk Also called **hard disk**. A form of nonvolatile secondary memory composed of rotating platters coated with a magnetic recording material. Because they are rotating mechanical devices, access times are about 5 to 20 milliseconds and cost per gigabyte in 2012 was \$0.05 to \$0.10.

main memory Also called **primary memory**. Memory used to hold programs while they are running; typically consists of DRAM in today's computers.

memory The storage area in which programs are kept when they are running, and that contains the data needed by the running programs.

memory hierarchy A structure that uses multiple levels of memories; as the distance from the processor increases, the size of the memories and the access time both increase.

message passing Communicating between multiple processors by explicitly sending and receiving information.

metastability A situation that occurs if a signal is sampled when it is not stable for the required setup and hold times, possibly causing the sampled value to fall into the indeterminate region between a high and low value.

microarchitecture The organization of the processor, including the major functional units, their interconnection, and control.

million instructions per second (MIPS) A measurement of program execution speed based on the number of millions of instructions. MIPS is computed as the instruction count divided by the product of the execution time and 10^6 .

MIMD or Multiple Instruction streams, Multiple Data streams. A multiprocessor.

minterms Also called **product terms**. A set of logic inputs joined by conjunction (AND operations); the product terms form the first logic stage of the programmable logic array (PLA).

miss penalty The time required to fetch a block into a level of the memory hierarchy from the lower level, including the time to access the block, transmit it from one level to the other, insert it in the level that experienced the miss, and then pass the block to the requestor.

miss rate The fraction of memory accesses not found in a level of the memory hierarchy.

most significant bit The leftmost bit in a LEGB8 doubleword.

multicore microprocessor A microprocessor containing multiple processors ("cores") in a single integrated circuit. Virtually all microprocessors today in desktops and servers are multicore.

multilevel cache A memory hierarchy with multiple levels of caches, rather than just a cache and main memory.

multiple issue A scheme whereby multiple instructions are launched in one clock cycle.

multiprocessor A computer system with at least two processors. This computer is in contrast to a uniprocessor, which has one, and is increasingly hard to find today.

multistage network A network that supplies a small switch at each node.

NAND gate An inverted AND gate.

network bandwidth Informally, the peak transfer rate of a network; can refer to the speed of a single link or the collective transfer rate of all links in the network.

next-state function A combinational function that, given the inputs and the current state, determines the next state of a finite-state machine.

nonblocking assignment An assignment that continues after evaluating the right-hand side, assigning the left-hand side the value only after all right-hand sides are evaluated.

nonblocking cache A cache that allows the processor to make references to the cache while the cache is handling an earlier miss.

nonuniform memory access (NUMA) A type of single address space multiprocessor in which some memory accesses are much faster than others depending on which processor asks for which word.

nonvolatile memory A form of memory that retains data even in the absence of a power source and that is used to store programs between runs. A DVD disk is nonvolatile.

nop An instruction that does no operation to change state.

NOR A logical bit-by-bit operation with two operands that calculates the NOT of the OR of the two operands. That is, it calculates a 1 only if there is a 0 in *both* operands.

NOR gate An inverted OR gate.

normalized A number in floating-point notation that has no leading 0s.

NOT A logical bit-by-bit operation with one operand that inverts the bits; that is, it replaces every 1 with a 0, and every 0 with a 1.

object oriented language A programming language that is oriented around objects rather than actions, or data versus logic.